



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

Ingeniería en Software

INFORME TÉCNICO

Plan de Pruebas Unitarias

Sistema AliGest — Gestión de Expensas Residenciales

Asignatura: Análisis y Diseño

NRC: 30723

Docente: Ing. Jenny Ruiz

Integrantes: Santi Jeancarlo

Erick Tufiño

Andrés Cedeño

Fecha: 09 de julio de 2026

Sangolquí — Ecuador

2026

Índice de Contenidos

1. Objetivo de las Pruebas	1
1.1. Objetivo General	1
1.2. Objetivos Específicos	1
2. Planteamiento	2
2.1. Alcance de las Pruebas	2
2.2. Enfoque de Testing	2
2.3. Estrategia de Mocking	3
2.4. Plan de Ejecución de Pruebas (Hoja de Ruta)	4
3. Herramientas y Entorno	7
3.1. Framework de Testing	7
3.2. Entorno de Pruebas	7
3.3. Estructura de Archivos	7
4. Implementación por Requisito	9
4.1. REQ001: Gestión de Acceso y Registro de Usuarios	9
4.2. REQ002: Recuperación de Contraseña	11
4.3. REQ003: Configuración de Perfiles	12
4.4. REQ004: Importación de Copropietarios	13
4.5. REQ005: Consulta de Copropietarios	15
4.6. REQ006: Generación de Reportes	16
4.7. REQ007: Modificación de Copropietarios	17
4.8. REQ008: Eliminación de Copropietarios	18
4.9. REQ009: Registro de Pago de Expensa	20
4.10. REQ010: Validación de Pagos	21
4.11. REQ011: Cálculo de Sanción por Mora	22
4.12. REQ012: Generación de Comprobante de Pago	25
4.13. REQ013: Notificaciones Automáticas de Pago	26
4.14. Pruebas Transversales de Modelo de Dominio y Objetos de Valor . . .	27
5. Resultados Obtenidos	29
5.1. Resumen de Ejecución	29
5.2. Desglose por Archivo	30
5.3. Métricas por Requisito	31
5.4. Distribución de Pruebas por Requisito	32

6. Problemas y Soluciones	33
6.1. Recalculo del Recargo por Mora	33
6.2. Ausencia de Generación de Deudas Mensuales	33
6.3. Redondeo de Punto Flotante en Cálculos Monetarios	33
6.4. Determinismo Temporal en Pruebas de Mora	34
6.5. Verificación de un Efecto Secundario Asíncrono en Segundo Plano . . .	34
6.6. Construcción de Datos de Prueba Válidos para el Algoritmo Módulo 10	34
6.7. Tiempo de Arranque de <code>mongodb-memory-server</code>	35
7. Conclusiones	36
7.1. Logros Alcanzados	36
7.2. Calidad del Software	36
7.3. Lecciones Aprendidas	36
7.4. Recomendaciones Futuras	37
7.5. Resumen Final	37
Anexos	39

Índice de tablas

1.	Hoja de ruta de pruebas planificadas por requisito	5
2.	Orden de ejecución sugerido por fase de la arquitectura	6
3.	Resumen global de ejecución de la suite de pruebas	29
4.	Desglose de pruebas por archivo	30
5.	Métricas de cobertura por requisito funcional	31
6.	Distribución porcentual de pruebas por requisito	32
7.	Distribución de pruebas por categoría arquitectónica	41
8.	Distribución de pruebas por tipo de validación	41

1. Objetivo de las Pruebas

1.1. Objetivo General

Validar que los requisitos funcionales del sistema **AliGest** —plataforma de gestión de expensas residenciales del conjunto habitacional “*La Primavera*”— cumplan con las especificaciones establecidas en el backlog del proyecto, garantizando el correcto funcionamiento de:

- Autenticación, registro y recuperación de acceso de usuarios.
- Configuración de perfiles y control de permisos por rol (Administrador / Copropietario).
- Administración de copropietarios: importación masiva desde Excel, consulta, modificación y eliminación controlada.
- Registro, validación y auditoría de pagos de expensas.
- Cálculo automático del recargo por mora sobre deudas vencidas.
- Generación de comprobantes digitales inmutables y notificaciones automáticas al residente.

1.2. Objetivos Específicos

1. **Validar objetos de valor y modelo de dominio:** verificar que *Cedula*, *Telefono* y *Usuario* apliquen correctamente sus reglas de negocio e invariantes (algoritmo Módulo 10, formato internacional ecuatoriano, políticas de contraseña).
2. **Validar servicios de negocio:** asegurar que *AuthService*, *CopropietarioService* y *PagoService* gestionen correctamente la lógica de negocio, las validaciones cruzadas contra la base de datos y el manejo de errores.
3. **Validar los patrones de diseño aplicados:** confirmar que el patrón *Strategy* (cálculo de mora), el patrón *Decorator* (auditoría de pagos) y el patrón *Adapter* (importación de Excel) se comporten según lo especificado.
4. **Validar middlewares de seguridad:** comprobar el control de acceso por rol, la protección del último administrador activo y la expiración de sesión por inactividad.
5. **Validar la integración con la capa de persistencia:** verificar que las operaciones sobre MongoDB (a través de `mongodb-memory-server`) sean correctas, aisladas entre pruebas y no dependan de una base de datos de producción.

2. Planteamiento

2.1. Alcance de las Pruebas

Las pruebas unitarias e de integración ligera cubren:

- **Objetos de valor y modelo de dominio:** validación de datos, invariantes y métodos de negocio (`Cedula`, `Telefono`, `Usuario`).
- **Servicios de negocio:** reglas de negocio, validaciones, manejo de errores y orquestación transaccional (`AuthService`, `CopropietarioService`, `PagoService`).
- **Patrones de diseño:** `EcuandorianMora12Strategy` / `CalculadorMoraContext` (`Strategy`), `AuditPaymentServiceDecorator` (`Decorator`) y `ExcelCopropietarioAdapter` (`Adapter`).
- **Middleware de presentación:** `verificarSesion`, `permitirSolo` y `verificarUltimoAdmini`
- **Controladores HTTP (integración):** `copropietarioController` mediante `supertest`, verificando el contrato completo petición-respuesta contra una base de datos en memoria.

Explícitamente **no** forman parte del alcance de este plan de pruebas automatizadas (se mencionan como trabajo futuro en la Sección 7.4):

- La interfaz visual (HTML/CSS/JavaScript del lado cliente en `dashboard.html` y `login.html`).
- La generación de los binarios PDF/Excel (`generarPdfLista`, `generarPdfReportePagos`, `generarPdfRecibo`, `generarExcelLista`), dado que su verificación exige comparación de contenido binario, no de valores lógicos.
- Pruebas de carga, rendimiento o end-to-end de navegador.

2.2. Enfoque de Testing

Metodología: desarrollo iterativo con pruebas de regresión (TDD adaptado) — cada corrección de comportamiento de negocio (por ejemplo, el recálculo del recargo por mora, ver Sección 6.2) fue acompañada de la actualización de su prueba correspondiente antes de dar por cerrado el cambio.

Tipo de pruebas: unitarias puras (con *mocks*) e integración ligera contra una instancia de MongoDB en memoria.

Cobertura objetivo: no existe un umbral de cobertura de código configurado en `jest.config.js`; en su lugar se verificó manualmente que cada requisito funcional del backlog (REQ001–REQ013) cuente con al menos un caso de prueba explícito, documentado en la Sección 4 y consolidado en la Tabla 5.

2.3. Estrategia de Mocking

AliGest combina **dos** estrategias de prueba complementarias, elegidas según el tipo de componente:

1. **Mocks explícitos de Jest (`jest.mock()`):** utilizados cuando el componente bajo prueba debe aislarse por completo de la capa de datos —middlewares de presentación y el decorador de auditoría—, ya que su responsabilidad es puramente de control de flujo, no de persistencia.
2. **Base de datos en memoria (`mongodb-memory-server`):** utilizada para los servicios de negocio (`AuthService`, `CopropietarioService`, `PagoService`) y para las pruebas de integración de controladores (`copropietarioController.test.js`), porque su lógica depende de consultas encadenadas reales (`populate`, unicidad de cedula/casa, amortización FIFO sobre colecciones relacionadas) que un mock manual reproduciría de forma incompleta o frágil.

Listing 1: Aislamiento con `jest.mock()` en `authMiddleware.test.js`

```
1 const { verificarSesion, permitirSolo, verificarUltimoAdministrador } =  
2   require('../../../../src/presentation/middlewares/authMiddleware');  
3 const usuarioRepository = require('../../../../src/data/repositories/  
4   usuarioRepository');  
5 jest.mock('../../../../src/data/repositories/usuarioRepository');
```

Listing 2: Integración real con `mongodb-memory-server` en `tests/setup.js`

```
1 const { MongoMemoryServer } = require('mongodb-memory-server');  
2 const mongoose = require('mongoose');  
3 const db = require('../config/database');  
4  
5 let mongoServer;  
6  
7 beforeAll(async () => {  
8   mongoServer = await MongoMemoryServer.create();  
9   const uri = mongoServer.getUri();  
10  process.env.MONGODB_URI = uri;  
11  await db.conectarBD();  
12 });  
13  
14 afterAll(async () => {  
15   await db.desconectarBD();  
16   if (mongoServer) {  
17     await mongoServer.stop();  
18   }  
19 });  
20  
21 beforeEach(async () => {
```

```
22     // Limpiar todas las colecciones antes de cada prueba (aislamiento entre casos
    )
23     await db.limpiarTablas();
24 });
```

2.4. Plan de Ejecución de Pruebas (Hoja de Ruta)

Antes de implementar los casos de prueba descritos en la Sección 4, se definió la siguiente hoja de ruta: un mapa de correspondencia entre cada requisito funcional del backlog, el componente de código que lo materializa y el archivo de prueba planificado para verificarlo. Esta planificación —reconstruida aquí a partir del trabajo ya ejecutado— resume el orden y el alcance que el lector debe esperar encontrar en el resto del informe.

Tabla 1: Hoja de ruta de pruebas planificadas por requisito

#	REQ	Componente / Capa	Archivo de prueba planificado	Tipo
1	REQ001	AuthService + authMiddleware + sessionRepository	AuthService.test.js / authMiddleware.test.js / authController.test.js / sessionRepository.test.js	Integración + Unitaria
2	REQ002	AuthService (recuperación)	AuthService.test.js	Integración
3	REQ003	authMiddleware + authController (perfiles)	authMiddleware.test.js / authController.test.js / sessionRepository.test.js	Unitaria + Integración
4	REQ004	CopropietarioService + excelAdapter	CopropietarioService.test.js / excelAdapter.test.js / ExcelImportSample.test.js	Integración + Unitaria
5	REQ005	copropietarioController	copropietarioController.test.js	Integración HTTP
6	REQ006	PagoService (reportes)	Cobertura indirecta (Sección 7.4)	Integración
7	REQ007	CopropietarioService (modificar)	CopropietarioService.test.js	Integración
8	REQ008	CopropietarioService + copropietarioController	CopropietarioService.test.js / copropietarioController.test.js	Integración
9	REQ009	PagoService + pagoController (reportar pago)	PagoService.test.js / pagoController.test.js	Integración
10	REQ010	PagoService + auditDecorator + pagoController	PagoService.test.js / auditDecorator.test.js / pagoController.test.js	Integración + Unitaria
11	REQ011	moraStrategy + PagoService (expensas)	moraStrategy.test.js / PagoService.test.js	Unitaria + Integración
12	REQ012	Pago	pagoController.test.js	Integración

Tabla 2: Orden de ejecución sugerido por fase de la arquitectura

Fase	Capa	Alcance
1	Dominio y Objetos de Valor	Validar invariantes de Cedula , Telefono y Usuario antes de probar cualquier servicio que dependa de ellos.
2	Patrones de Diseño	Verificar Strategy (mora), Adapter (Excel) y Decorator (auditoría) de forma aislada.
3	Servicios de Negocio	Probar AuthService , CopropietarioService y PagoService contra mongodb-memory-server , una vez validadas sus dependencias de dominio y patrones.
4	Presentación	Cerrar con los <i>middlewares</i> de seguridad y las pruebas de integración HTTP sobre los controladores.

Como se detalla en la Sección 4, de las 13 líneas planificadas, 12 se implementaron con cobertura completa y 1 (REQ006, generación de reportes) quedó con cobertura parcial —la restricción por rol se ejercita de forma indirecta mediante las pruebas de integración de controladores, pero `obtenerReportePagos/generarPdfReportePagos` aún no tienen un caso dedicado—; esa brecha se documenta de forma explícita en la Sección 7.4 (Recomendaciones Futuras) en lugar de omitirse.

3. Herramientas y Entorno

3.1. Framework de Testing

Jest 30.4.2 — framework de pruebas de JavaScript, ejecutado sobre Node.js en modo CommonJS (`require/module.exports`), sin necesidad de transpilación.

3.2. Entorno de Pruebas

- **Node.js:** entorno de ejecución `testEnvironment: 'node'` (no se requiere DOM, ya que la capa de presentación validada es HTTP, no navegador).
- **Base de datos:** `mongodb-memory-server 11.2.0`, instancia efímera de MongoDB que se crea antes de la suite y se destruye al finalizar.
- **Peticiones HTTP simuladas:** `supertest 7.2.2`, para ejercer la aplicación Express completa (`app.js`) sin necesidad de abrir un puerto real.
- **Ejecución:** `jest -runInBand` (definido en `package.json`), en serie para evitar condiciones de carrera sobre la misma base de datos en memoria compartida entre archivos de prueba.

3.3. Estructura de Archivos

Listing 3: Estructura del directorio `tests/`

```
1 tests/
2   setup.js
3   core/
4     business/
5       patterns/
6         auditDecorator.test.js (2 tests)
7         excelAdapter.test.js (4 tests)
8         moraStrategy.test.js (4 tests)
9       services/
10        AuthService.test.js (9 tests)
11        CopropietarioService.test.js(7 tests)
12        ExcellImportSample.test.js (1 test)
13        NotificationService.test.js (3 tests)
14        PagoService.test.js (10 tests)
15      domain/
16      models/
17        Usuario.test.js (7 tests)
18      valueObjects/
19        Cedula.test.js (7 tests)
20        Telefono.test.js (7 tests)
21      data/
22      repositories/
```

```
23   sessionRepository.test.js (3 tests)
24 presentation/
25   controllers/
26     authController.test.js (4 tests)
27     copropietarioController.test.js (5 tests)
28     pagoController.test.js (4 tests)
29   middlewares/
30     authMiddleware.test.js (6 tests)
31   views/
32     dashboard.test.js (3 tests)
```

4. Implementación por Requisito

Esta sección documenta, para cada requisito funcional (REQ001–REQ013, según la numeración consolidada en el backlog del proyecto), el código relevante que implementa la funcionalidad, el archivo de prueba correspondiente y los resultados obtenidos.

4.1. REQ001: Gestión de Acceso y Registro de Usuarios

Archivos: `AuthService.test.js` (login, registro, bloqueo y desbloqueo), `authMiddleware.test.js` (verificarSesion/permitirSolo), `authController.test.js` (registro público bloqueado) y `sessionRepository.test.js` (sesión aleatoria y expiración).

Total de pruebas: 12 — **Estado:** 12/12 PASANDO (100 %).

Cómo se implementó

El registro y el inicio de sesión se validan primero contra las reglas de dominio del objeto Usuario y luego contra el repositorio de persistencia:

Listing 4: `AuthService.login` — validación de credenciales y bloqueo temporal (`src/core/business/services/AuthService.js`)

```
1 async login(username, password) {
2   if (!username || !password) {
3     throw new Error("El usuario y la contraseña son obligatorios.");
4   }
5
6   const user = await usuarioRepository.findByUsername(username);
7   if (!user) {
8     throw new Error("Nombre de usuario o contraseña incorrectos. Intente nuevamente.");
9   }
10
11   const domainUser = new Usuario(user);
12   if (domainUser.esBloqueado()) {
13     const limite = new Date(user.lockout_until);
14     const minRestantes = Math.ceil((limite - new Date()) / (1000 * 60));
15     throw new Error('Cuenta bloqueada temporalmente por mltiples intentos fallidos. Intente en ${minRestantes} minutos o recupere su contraseña.');
16   }
17
18   const valid = await bcrypt.compare(password, user.password);
19   if (!valid) {
20     await usuarioRepository.incrementFailedAttempts(username);
21     const updatedUser = await usuarioRepository.findByUsername(username);
22     if (updatedUser.failed_attempts >= 3) {
```

```

23     const lockoutTime = new Date(Date.now() + 15 * 60 * 1000).toISOString()
24     ;
25     await usuarioRepository.lockoutUser(username, lockoutTime);
26     throw new Error("Cuenta bloqueada temporalmente por mltiples intentos
27         fallidos. Intente en 15 minutos o recupere su contrasea.");
28 }
29 const intentosRestantes = 3 - updatedUser.failed_attempts;
30 throw new Error('Nombre de usuario o contrasea incorrectos. Le quedan ${
31     intentosRestantes} intentos antes de bloquear la cuenta.');
```

El middleware `verificarSesion` controla, además del rol, la expiración de sesión por inactividad (RNF-01, ver Sección 4.11 y el detalle del mecanismo en el Anexo C):

Listing 5: Prueba de bloqueo por 3 intentos fallidos (`AuthService.test.js`)

```

1 test('Debera bloquear la cuenta por 15 minutos en el tercer intento fallido de
2   inicio de sesin', async () => {
3     const username = 'bloqueable';
4     const passwordCorrecta = 'Password123!';
5     await authService.registrarUsuarioPublico(username, 'bloqueo@correo.com',
6       passwordCorrecta);
7
8     await expect(authService.login(username, 'Incorrecta')).rejects.toThrow("Le
9       quedan 2 intentos");
10    await expect(authService.login(username, 'Incorrecta')).rejects.toThrow("Le
11      quedan 1 intentos");
12    await expect(authService.login(username, 'Incorrecta'))
13      .rejects.toThrow("Cuenta bloqueada temporalmente por mltiples intentos
14        fallidos");
15
16    await expect(authService.login(username, passwordCorrecta))
17      .rejects.toThrow("Cuenta bloqueada temporalmente");
18  });
```

Resultados Obtenidos

- Registro público con contraseña cifrada (`bcrypt`, factor 12) — verificado.
- Rechazo de usuario/correo duplicado — verificado.
- Inicio de sesión válido con carga de permisos por rol — verificado.
- Bloqueo progresivo (2 intentos restantes → 1 → bloqueo de 15 minutos) y persistencia del bloqueo ante credenciales correctas — verificado.

- Autorización por encabezados `x-user-role/x-user-id` y rechazo de roles inválidos (401/403) — verificado.
- Restricción de rutas por rol mediante `permitirSolo(...)` — verificado.

4.2. REQ002: Recuperación de Contraseña

Archivo: `tests/core/business/services/AuthService.test.js` (4 casos: generación del OTP, restablecimiento, expiración del código e invalidación tras 3 intentos).

Total de pruebas: 4 — **Estado:** 4/4 PASANDO (100 %).

Cómo se implementó

Listing 6: Generación y expiración del código OTP (`AuthService.js`)

```
1 async generarCodigoRecuperacion(email) {
2   if (!email) {
3     throw new Error("El correo electrónico es obligatorio para recuperar
4       contraseña.");
5   }
6   const user = await usuarioRepository.findByEmail(email);
7   if (!user) {
8     throw new Error("El correo electrónico ingresado no está registrado en el
9       sistema. Contacte al administrador.");
10  }
11  const codigo = Math.floor(100000 + Math.random() * 900000).toString();
12  const fechaExpiracion = new Date(Date.now() + 10 * 60 * 1000); // vigencia de
13    10 minutos
14  await usuarioRepository.updateRecoveryCode(user.username, codigo,
15    fechaExpiracion.toISOString());
16  // ... simulación de despacho por SMTP (consola) ...
17  return { mensaje: "Código enviado correctamente.", codigoSimulado: codigo,
18    username: user.username };
19 }
```

Listing 7: Prueba de expiración del código OTP (`AuthService.test.js`)

```
1 test('Debe fallar al cambiar clave si el código OTP ha expirado', async () => {
2   const username = 'expirable';
3   const email = 'expirable@correo.com';
4   await authService.registrarUsuarioPublico(username, email, 'OldPassword123!');
5
6   await authService.generarCodigoRecuperacion(email);
7   const expiracionPasada = new Date(Date.now() - 10 * 1000).toISOString();
8   const user = await usuarioRepository.findByUsername(username);
9   await usuarioRepository.updateRecoveryCode(username, user.recovery_code,
10     expiracionPasada);
11 }
```

```
10
11     await expect(authService.recuperarContraseña(username, user.recovery_code, '
12         NewPassword123!'))
13         .rejects.toThrow("El código de verificación ha expirado. Solicite uno nuevo
        .");
    });
```

Resultados Obtenidos

- Generación de código de 6 dígitos con vigencia de 10 minutos — verificado.
- Restablecimiento exitoso invalida la contraseña anterior — verificado.
- Rechazo de código expirado con mensaje explícito — verificado.

4.3. REQ003: Configuración de Perfiles

Archivos: `authMiddleware.test.js` (protección del último administrador), `authController.test.js` (promoción de perfil, listado con nombre/casa y bloqueo de degradación del único administrador) y `sessionRepository.test.js` (invalidación de sesiones al cambiar el perfil).

Total de pruebas: 6 — **Estado:** 6/6 PASANDO (100 %).

Cómo se implementó

Listing 8: `verificarUltimoAdministrador` (`authMiddleware.js`)

```
1 async function verificarUltimoAdministrador(req, res, next) {
2     const { id } = req.params;
3     if (!id) return next();
4
5     const usuarioTarget = await usuarioRepository.findById(id);
6     if (!usuarioTarget) {
7         return res.status(404).json({ error: "El usuario especificado no existe en
            el sistema." });
8     }
9
10    if (usuarioTarget.role === 'ADMINISTRADOR') {
11        const numAdmins = await usuarioRepository.countAdministrators();
12        if (numAdmins <= 1) {
13            return res.status(400).json({
14                error: "Operación de seguridad denegada: No se puede eliminar o
                    desactivar al último administrador del sistema."
15            });
16        }
17    }
18    next();
```



```
19 }
```

Listing 9: Prueba del bloqueo al último administrador activo (`authMiddleware.test.js`)

```
1 test('Debera bloquear la accin (400) si se intenta eliminar o desactivar al ltimo
  administrador activo del sistema', async () => {
2   mockReq.params.id = 'ultimo-admin';
3   usuarioRepository.findById.mockResolvedValue({ id: 'ultimo-admin', role: '
    ADMINISTRADOR' });
4   usuarioRepository.countAdministrators.mockResolvedValue(1); // Solo queda 1
    admin
5
6   await verificarUltimoAdministrador(mockReq, mockRes, nextFunction);
7
8   expect(mockRes.status).toHaveBeenCalledWith(400);
9   expect(mockRes.json).toHaveBeenCalledWith(expect.objectContaining({
10    error: expect.stringContaining("No se puede eliminar o desactivar al ltimo
      administrador")
11  }));
12   expect(nextFunction).not.toHaveBeenCalled();
13 });
```

Resultados Obtenidos

- Modificación de perfiles de usuarios distintos de ADMINISTRADOR sin restricción — verificado.
- Degradación de un ADMINISTRADOR permitida cuando existen otros administradores activos — verificado.
- Bloqueo (400) cuando la operación dejaría al sistema sin ningún administrador — verificado.

4.4. REQ004: Importación de Copropietarios

Archivos: `CopropietarioService.test.js` (3 casos: creación manual e importación masiva), `excelAdapter.test.js` (4 casos) y `ExcelImportSample.test.js` (importación aislada de 60 filas con creación de cuentas).

Total de pruebas: 8 — **Estado:** 8/8 PASANDO (100 %).

Cómo se implementó

El patrón *Adapter* tolera variaciones de encabezado del archivo Excel de origen:

Listing 10: `ExcelCopropietarioAdapter.adaptRow` (`src/core/business/patterns/excelAdapter.js`)

```
1 static adaptRow(externalRow) {
2   if (!externalRow) {
```

```

3      throw new Error("Fila de hoja de cálculo nula o indefinida.");
4    }
5    const cedula = externalRow["Cedula"] || externalRow["Cedula"] || externalRow["ID_Nacional"] || "";
6    const nombre = externalRow["Nombre Completo"] || externalRow["Nombre"] || "";
7    const casa = externalRow["Número Casa"] || externalRow["Casa"] || "";
8    const telefono = externalRow["Teléfono"] || externalRow["Teléfono"] || externalRow["Celular"] || "";
9    const email = externalRow["Correo Electrónico"] || externalRow["Correo"] || externalRow["Email"] || "";
10   const saldo = parseFloat(externalRow["Saldo Inicial"] || externalRow["Saldo"] || 0.0);
11
12   return {
13     cedula: String(cedula).trim(), nombre: String(nombre).trim(), casa: String(casa).trim(),
14     telefono: String(telefono).trim(), email: String(email).trim(),
15     saldo: isNaN(saldo) ? 0.0 : saldo
16   };
17 }

```

La importación masiva es **atómica**: valida todo el lote antes de persistir cualquier registro, garantizando que un solo error cancele la operación completa (REQ004):

Listing 11: Prueba de atomicidad de la importación masiva (CopropietarioService.test.js)

```

1 test('Debera abortar toda la importación masiva si existe una sola fila inválida (Atomicidad)', async () => {
2   const filas = [
3     { "Cedula": "1721151247", "Nombre Completo": "Juan Perez", "Número Casa": "Villa 1", "Correo": "juan@correo.com" },
4     { "Cedula": "1721151241", "Nombre Completo": "Fila Inválida", "Número Casa": "Villa 2", "Correo": "invalido" }
5   ];
6   const buffer = crearExcelMock(filas);
7
8   await expect(copropietarioService.importarMasivoDesdeExcel(buffer))
9     .rejects.toThrow("Importación cancelada por errores de validación");
10
11   const todos = await copropietarioService.obtenerTodos();
12   expect(todos).toHaveLength(0); // Nada se persistió
13 });

```

Resultados Obtenidos

- Registro manual individual con generación automática de usuario/contraseña temporal (`must_change_password = 1`) — verificado.
- Importación masiva exitosa de 2 filas con generación de `username` a partir del número de casa — verificado.
- Cancelación total del lote ante una fila con cédula/correo inválidos — verificado.
- Mapeo tolerante de encabezados en español, alternativos y valores de saldo no numéricos — verificado (`excelAdapter.test.js`).

4.5. REQ005: Consulta de Copropietarios

Archivo: `tests/presentation/controllers/copropietarioController.test.js`
(3 casos, integración HTTP con `supertest`: listado autenticado, filtro por nombre y restricción de la nómina//me para el rol Copropietario).

Total de pruebas: 3 — **Estado:** 3/3 PASANDO (100 %).

Cómo se implementó

Listing 12: Filtro por nombre en GET `/api/copropietarios`
(`src/presentation/controllers/copropietarioController.js`)

```
1 router.get('/', verificarSesion, async (req, res) => {
2   const { nombre, casa, casaInicio, casaFin, estadoCuenta } = req.query;
3   let lista = await copropietarioService.obtenerTodos();
4
5   if (nombre) {
6     const queryName = nombre.toLowerCase().trim();
7     lista = lista.filter(c => c.nombre.toLowerCase().includes(queryName));
8   }
9   // ... filtros adicionales por casa, rango de villas y estado de cuenta ...
10  res.status(200).json(lista);
11 });
```

Listing 13: Prueba de integración con `supertest` (`copropietarioController.test.js`)

```
1 test('GET /api/copropietarios - Debera filtrar residentes por nombre', async ()
2   => {
3     await Copropietario.create({
4       cedula: '0921151247', nombre: 'Maria Gomez', casa: 'Villa 20',
5       telefono: '+593987654322', email: 'maria@correo.com', saldo: 0.0
6     });
7
8     const response = await request(app)
9       .get('/api/copropietarios?nombre=Maria')
10      .set(adminTokenHeaders);
```

```
10
11     expect(response.status).toBe(200);
12     expect(response.body).toHaveLength(1);
13     expect(response.body[0].nombre).toBe('Maria Gomez');
14 });
```

Resultados Obtenidos

- Listado completo de copropietarios autenticados (rol Administrador) — verificado.
- Filtro por nombre insensible a mayúsculas — verificado.

Nota de Cobertura

Los filtros por rango de villas, estado de cuenta (**saldado/deudor**) y el ordenamiento de columnas se implementan en el mismo controlador, pero no cuentan con un caso de prueba automatizado dedicado; fueron verificados manualmente durante el desarrollo. Se recomienda ampliar `copropietarioController.test.js` con estos escenarios (ver Sección 7.4).

4.6. REQ006: Generación de Reportes

Total de pruebas: 0 — **Estado:** SIN COBERTURA AUTOMATIZADA DEDICADA.

Cómo se implementó

`PagoService.obtenerReportePagos` restringe automáticamente el conjunto de resultados según el rol del usuario autenticado:

Listing 14: Restricción de reportes por rol (`src/core/business/services/PagoService.js`)

```
1  async obtenerReportePagos(filtros, userContext) {
2      const role = userContext?.role;
3      let copropietarioId = null;
4
5      if (role === 'COPROPIETARIO') {
6          const copro = await copropietarioRepository.findByUsuarioId(userContext.id);
7          if (!copro) return [];
8          copropietarioId = copro.id;
9      }
10
11      return await pagoRepository.getReportePagos({
12          fechaInicio: filtros.fechaInicio || null,
13          fechaFin: filtros.fechaFin || null,
```

```
14     periodo: filtros.periodo || null,  
15     estado: filtros.estado || null,  
16     copropietarioId  
17   });  
18 }
```

Resultados Obtenidos y Nota de Cobertura

La lógica de restricción por rol reutiliza el mismo mecanismo ya validado indirectamente en `PagoService.test.js` (búsqueda de copropietario por `usuarioId`), pero no existe, al cierre de este informe, un caso de prueba que invoque `obtenerReportePagos` ni `generarPdfReportePagos` directamente. Se identifica como una brecha de cobertura y se registra como recomendación prioritaria en la Sección 7.4.

4.7. REQ007: Modificación de Copropietarios

Archivo: `tests/core/business/services/CopropietarioService.test.js` (3 casos).

Total de pruebas: 3 — **Estado:** 3/3 PASANDO (100 %).

Cómo se implementó

Además de la actualización estándar de datos de contacto, `actualizarDatos` permite el cambio de cédula (representante nuevo de la vivienda) regenerando automáticamente la contraseña temporal de acceso:

Listing 15: Regeneración de contraseña ante cambio de cédula (`CopropietarioService.js`)

```
1  if (datos.cedula) {  
2    const cedulaSolicitada = new Cedula(datos.cedula).valor;  
3    if (cedulaSolicitada !== copropietario.cedula) {  
4      const existeCedula = await copropietarioRepository.findByCedula(  
5        cedulaSolicitada);  
6      if (existeCedula) {  
7        throw new Error('La cedula ${cedulaSolicitada} ya est registrada en el  
8          sistema.');9      }  
10     nuevaCedula = cedulaSolicitada;  
11  
12     if (copropietario.usuario_id) {  
13       passwordTemporal = `Temp-${Math.floor(1000 + Math.random() * 9000)}!`;   
14       const hashed = await bcrypt.hash(passwordTemporal, 12);  
15       await usuarioRepository.updatePasswordAndForceMustChange(copropietario.  
16         usuario_id, hashed);  
17     }  
18   }
```

```
15 }
16 }
```

Listing 16: Prueba del cambio de representante (CopropietarioService.test.js)

```
1 test('RF-2.4: Debera permitir cambiar la cedula (nuevo representante) y regenerar
  la contraseña temporal', async () => {
2   await copropietarioService.crearCopropietario({
3     cedula: '1721151247', nombre: 'Juan Original', casa: 'Villa 2',
4     telefono: '0987654321', email: 'juan2@correo.com'
5   });
6
7   const actualizado = await copropietarioService.actualizarDatos('1721151247', {
8     cedula: '3000000012', nombre: 'Pedro Nuevo Representante'
9   });
10
11   expect(actualizado.cedula).toBe('3000000012');
12   expect(actualizado.passwordTemporal).toEqual(expect.stringMatching(/~Temp-\d
    {4}!$/));
13
14   const anterior = await copropietarioRepository.findByCedula('1721151247');
15   expect(anterior).toBeNull();
16
17   const nuevo = await copropietarioRepository.findByCedula('3000000012');
18   const usuario = await usuarioRepository.findById(nuevo.usuario_id);
19   expect(usuario.must_change_password).toBe(1);
20 });
```

Resultados Obtenidos

- Actualización de nombre, villa, teléfono y correo — verificado.
- Cambio de cédula con regeneración de contraseña temporal y forzado de cambio en el primer ingreso — verificado.
- Rechazo del cambio si la nueva cédula ya pertenece a otro copropietario — verificado.

4.8. REQ008: Eliminación de Copropietarios

Archivos: CopropietarioService.test.js (1 caso) y copropietarioController.test.js (2 casos, integración HTTP).

Total de pruebas: 3 — **Estado:** 3/3 PASANDO (100 %).

Cómo se implementó

El controlador exige confirmación explícita cuando el copropietario tiene historial de pagos, y bloquea por completo la eliminación si tiene deudas pendientes:

Listing 17: Confirmación por historial de pagos (copropietarioController.js)

```
1 router.delete('/:id', verificarSesion, permitirSolo('ADMINISTRADOR'), async (req,
  res) => {
2   const id = req.params.id;
3   const { confirmar } = req.query;
4
5   const deudasPendientes = await pagoRepository.countPendingDeudas(id);
6   if (deudasPendientes > 0) {
7     return res.status(400).json({ error: "No se permite eliminar un residente
      con deudas pendientes." });
8   }
9
10  const tienePagos = await pagoRepository.countPayments(id);
11  if (tienePagos > 0 && confirmar !== 'true') {
12    return res.status(202).json({
13      advertencia: "HISTORIAL_ACTIVO",
14      mensaje: "ADVERTENCIA: Este copropietario tiene historial de pagos
      registrado. Eliminar este registro puede afectar reportes historicos
      y estadsticas. Desea continuar de todas formas?"
15    });
16  }
17  const result = await copropietarioService.eliminarCopropietario(id);
18  res.status(200).json(result);
19 });
```

Listing 18: Prueba de la advertencia 202 por historial activo (copropietarioController.test.js)

```
1 test('DELETE /api/copropietarios/:id - Debera denegar con 202 y advertencia si
  tiene historial de pagos sin confirmar', async () => {
2   await Pago.create({
3     comprobante_id: 'DEP-777', copropietario_id: coproId, monto_pagado: 100.0,
4     estado: 'APROBADO', fecha_registro: new Date().toISOString()
5   });
6
7   const response = await request(app)
8     .delete(`/api/copropietarios/${coproId}`)
9     .set(adminTokenHeaders);
10
11  expect(response.status).toBe(202);
12  expect(response.body.advertencia).toBe('HISTORIAL_ACTIVO');
13 });
```

Resultados Obtenidos

- Bloqueo total de la eliminación si existen deudas pendientes — verificado.
- Advertencia (202) si existe historial de pagos sin confirmación explícita — verificado.
- Eliminación (borrado lógico del copropietario y físico del usuario) al confirmar con `?confirmar=true` — verificado.

4.9. REQ009: Registro de Pago de Expensa

Archivos: `PagoService.test.js` (2 casos: reporte pendiente y rechazo de monto no positivo) y `pagoController.test.js` (2 casos de integración HTTP: conservación de la imagen del comprobante y rechazo de un archivo que finge ser PNG).

Total de pruebas: 4 — **Estado:** 4/4 PASANDO (100 %).

Cómo se implementó

Listing 19: `reportarPagoCopropietario` (`PagoService.js`)

```
1 async reportarPagoCopropietario(usuarioId, comprobanteId, monto, metodo, periodo,
2   comprobanteImg) {
3   const copro = await copropietarioRepository.findByUsuarioId(usuarioId);
4   if (!copro) {
5     throw new Error("No se encontr el registro de copropietario para el
6       usuario actual.");
7   }
8
9   const pagoEntidad = new Pago({ copropietarioId: copro.id, monto, comprobanteId
10     , metodo, periodo, comprobanteImg });
11   if (!pagoEntidad.esMontoValido()) {
12     throw new Error("El monto reportado debe ser un nmero positivo.");
13   }
14
15   await pagoRepository.registrarPagoInmutable(
16     pagoEntidad.comprobanteId, pagoEntidad.copropietarioId, pagoEntidad.monto,
17     'PENDIENTE_VALIDACION', null, pagoEntidad.metodo, pagoEntidad.periodo,
18     pagoEntidad.comprobanteImg
19   );
20
21   return { mensaje: "El pago fue reportado con xito y est pendiente de
22     verificacin." };
23 }
```


Resultados Obtenidos

- Reporte de pago queda en estado `PENDIENTE_VALIDACION` con la referencia bancaria y el monto correctos — verificado.
- Rechazo de montos no positivos (`esMontoValido()`) — verificado.

Nota de Cobertura

La validación de tipo de archivo (JPG/JPEG/PNG) y del límite de 5 MB del comprobante se implementa mediante `multer` directamente en la capa de controlador (`pagoController.js`); al depender del *middleware* de carga multipart, no se ejerce en las pruebas de servicio aquí descritas y queda pendiente como prueba de integración HTTP futura (Sección 7.4).

4.10. REQ010: Validación de Pagos

Archivos: `PagoService.test.js` (reserva atómica anti-doble aprobación y rechazo con motivo), `auditDecorator.test.js` (2 casos del patrón *Decorator*) y `pagoController.test.js` (rechazo vía API con el `ObjectId` del pago).

Total de pruebas: 5 — **Estado:** 5/5 PASANDO (100 %).

Cómo se implementó

La aprobación de un pago se envuelve con el patrón *Decorator* para registrar auditoría sin acoplar esa responsabilidad a la lógica de negocio:

Listing 20: `AuditPaymentServiceDecorator` (`src/core/business/patterns/auditDecorator.js`)

```
1 function AuditPaymentServiceDecorator(baseService) {
2   return {
3     async procesarPago(pagoId) {
4       await pagoRepository.registrarAuditoria("INTENTO_APROBACION_PAGO", {
5         pagoId, timestamp: new Date().toISOString() });
6       try {
7         const resultado = await baseService.validarAprobarPago(pagoId);
8         await pagoRepository.registrarAuditoria("PAGO_APROBADO_EXITO", {
9           pagoId, reciboId: resultado.comprobante_id, monto: resultado.
10             monto_aplicado,
11             sobrepago: resultado.sobrepago, timestamp: new Date().
12               toISOString()
13             });
14         return resultado;
15       } catch (err) {
16         await pagoRepository.registrarAuditoria("APROBACION_PAGO_FALLIDA",
17           { pagoId, error: err.message, timestamp: new Date().toISOString()
18             });
19       }
20     }
21   };
22 }
```

```
14         throw err;
15     }
16 }
17 };
18 }
```

Listing 21: Prueba de auditoría en caso de fallo (`auditDecorator.test.js`, con `jest.mock`)

```
1 test('Debera registrar auditora de fallo si el servicio base lanza un error',
  async () => {
2     const mockError = new Error("Saldo insuficiente");
3     mockBaseService.validarAprobarPago.mockRejectedValue(mockError);
4     pagoRepository.registrarAuditoria.mockResolvedValue('AUDIT-ID');
5
6     await expect(decoratedService.procesarPago('PAGO-123')).rejects.toThrow("Saldo
      insuficiente");
7
8     expect(pagoRepository.registrarAuditoria).toHaveBeenCalledTimes(2);
9     expect(pagoRepository.registrarAuditoria).toHaveBeenNthCalledWith(
10         2, "APROBACION_PAGO_FALLIDA",
11         expect.objectContaining({ pagoId: 'PAGO-123', error: "Saldo insuficiente"
12             })
13     );
14 });
```

Resultados Obtenidos

- Aprobación de pago con amortización FIFO, generación de recibo digital y auditoría de éxito (`INTENTO_APROBACION_PAGO` → `PAGO_APROBADO_EXITO`) — verificado.
- Auditoría de fallo (`APROBACION_PAGO_FALLIDA`) cuando el servicio base lanza una excepción — verificado.
- Rechazo de pago con motivo obligatorio y notificación al residente — verificado.

4.11. REQ011: Cálculo de Sanción por Mora

Archivos: `tests/core/business/patterns/moraStrategy.test.js` (4 casos) y `PagoService.test.js` (6 casos: amortización FIFO con mora, no reapiación del 12 %, generación de expensas mensuales, no duplicación por período, validación de formato y vencimiento con cambio de año — infraestructura necesaria para que exista una deuda real que amortizar).

Total de pruebas: 10 — **Estado:** 10/10 PASANDO (100 %).

Cómo se implementó

El recargo por mora se implementa con el patrón *Strategy*, como un **recargo fijo del 12 %** aplicado una única vez por período vencido (no se prorratea por días de atraso ni se acumula):

Listing 22: EcuandorianMora12Strategy (src/core/business/patterns/moraStrategy.js)

```
1 class CalculadorMoraContext {
2   constructor(strategy) { this.strategy = strategy; }
3   calcular(monto, diasRetraso) { return this.strategy.calcularMora(monto,
4     diasRetraso); }
5 }
6 class EcuandorianMora12Strategy {
7   calcularMora(monto, diasRetraso) {
8     if (diasRetraso <= 0) return 0;
9     // Recargo fijo del 12% sobre la deuda vencida, aplicado una sola
10    // vez por periodo (no se prorratea ni se acumula por das de atraso).
11    const interes = monto * 0.12;
12    return parseFloat(interres.toFixed(2));
13  }
14 }
```

Listing 23: Prueba de invariancia del recargo respecto a los días de atraso (moraStrategy.test.js)

```
1 test('El recargo del 12% no debe acumularse ni variar segn los das de retraso
2   transcurridos', () => {
3   // El recargo se aplica una sola vez por periodo vencido (RF-3.3),
4   // por lo que el resultado es el mismo con 1 da o con 200 das de mora.
5   expect(context.calcular(100, 1)).toBe(12.00);
6   expect(context.calcular(100, 200)).toBe(12.00);
7 });
```

Dado que la generación de la deuda mensual (Deuda) es un prerequisite para que el cálculo de mora tenga un caso real que amortizar, se añadió y probó **generarExpensasMensuales**, que registra la expensa del período para todos los copropietarios sin duplicar períodos ya generados, con vencimiento el día 5 del mes siguiente:

Listing 24: generarExpensasMensuales (PagoService.js)

```
1 function calcularVencimientoExpensa(periodo) {
2   const [anioStr, mesStr] = periodo.split('-');
3   let anio = parseInt(anioStr, 10);
4   let mes = parseInt(mesStr, 10) + 1; // mes (1-12) del periodo + 1 = mes
5   siguiente
6   if (mes > 12) { mes = 1; anio += 1; }
```

```

6     return `${anio}-${String(mes).padStart(2, '0')}-05`;
7 }
8
9 async generarExpensasMensuales(periodo, montoBase) {
10     if (!periodo || !/^\d{4}-(0[1-9]|1[0-2])$/.test(periodo)) {
11         throw new Error("El periodo debe tener el formato YYYY-MM.");
12     }
13     const monto = parseFloat(montoBase);
14     const fechaVencimiento = calcularVencimientoExpensa(periodo);
15     const copropietarios = await copropietarioRepository.findAll();
16
17     let generadas = 0, omitidas = 0;
18     for (const copro of copropietarios) {
19         const existente = await pagoRepository.findDeudaPorPeriodo(copro.id,
20             periodo);
21         if (existente) { omitidas++; continue; }
22         await pagoRepository.createDeuda(copro.id, periodo, monto, 'PENDIENTE',
23             fechaVencimiento);
24         generadas++;
25     }
26     return { mensaje: `Expensas del periodo ${periodo} generadas correctamente.`,
27         periodo, montoBase: monto, fechaVencimiento, generadas, omitidas,
28         totalCopropropietarios: copropietarios.length };
29 }

```

Listing 25: Amortización FIFO con recargo del 12% sobre la deuda vencida (PagoService.test.js)

```

1 test('Debera aplicar amortizacin FIFO e inters de mora del 12% al aprobar un pago
2     ', async () => {
3     // Deuda 1: Vencida hace 10 das, monto $100. Deuda 2: Vigente, monto $100.
4     const d1Id = await pagoRepository.createDeuda(copro.id, '2026-06', 100.00, '
5         PENDIENTE', '2026-07-10');
6     const d2Id = await pagoRepository.createDeuda(copro.id, '2026-07', 100.00, '
7         PENDIENTE', '2026-07-30');
8
9     await pagoService.reportarPagoCopropietario(usuarioId, 'TRANS-FIFO', 150.00, '
10         TRANSFERENCIA', '2026-07', 'img');
11     const pagoId = (await pagoService.obtenerPagosPendientes())[0].id;
12
13     const resultado = await pagoService.validarAprobarPago(pagoId, new Date
14         ('2026-07-20'));
15     expect(resultado.estado_transaccion).toBe('APROBADO_E_INMUTABLE');
16
17     // Deuda 1: 100 + (100*0.12) = 112.00 -> se cancela por completo (150 - 112 =
18         38 restante)
19     // Deuda 2: sin mora (vigente) -> 100 - 38 = 62.00 pendiente

```

```

14   const deudas = await pagoRepository.getDeudasByCopropietario(copro.id);
15   expect(deudas.find(d => d.id === d1Id).estado).toBe('PAGADO');
16   expect(deudas.find(d => d.id === d2Id).monto).toBe(62.00);
17 });

```

Resultados Obtenidos

- Recargo cero para deudas no vencidas — verificado.
- Recargo fijo del 12 % sobre el monto adeudado, sin prorrateo diario — verificado.
- Invariancia del recargo frente a distintos períodos de atraso (1 día vs. 200 días) — verificado.
- Amortización FIFO completa: cancelación total de la deuda más antigua y aplicación del remanente a la siguiente — verificado.
- Generación de expensas mensuales sin duplicar el período y con vencimiento correcto (incluido el cambio de año en diciembre → enero) — verificado.

4.12. REQ012: Generación de Comprobante de Pago

Archivo: tests/presentation/controllers/pagoController.test.js (1 caso de integración HTTP).

Total de pruebas: 1 — **Estado:** 1/1 PASANDO (100 %).

Cómo se implementó

Listing 26: Pago.generarComprobanteDigitalID (src/core/domain/models/Pago.js)

```

1  generarComprobanteDigitalID(casa, numeroSecuencial) {
2      const hoy = new Date();
3      const anio = hoy.getFullYear();
4      const mes = String(hoy.getMonth() + 1).padStart(2, '0');
5      const casaSanitizada = casa.replace(/[^a-zA-Z0-9]/g, '').toUpperCase();
6      const secuencia = String(numeroSecuencial).padStart(4, '0');
7      this.reciboId = `AGE-${anio}-${mes}-${casaSanitizada}-${secuencia}`;
8      return this.reciboId;
9  }

```

Resultados Obtenidos

La prueba *RF-3.2/RF-3.4: permite aprobar desde la API y genera recibo inmutable* de pagoController.test.js aprueba un pago vía POST /api/pagos/:id/aprobar y verifica que se genere el identificador de recibo con el formato AGE-YYYY-MM-CASA-SEQ y que el pago quede en estado APROBADO e inmutable. El formato también se corrobora dentro de la prueba de amortización FIFO de la Sección 4.11. La verificación binaria

byte a byte del PDF (`generarPdfRecibo`) queda fuera del alcance automatizado por tratarse de comparación de contenido binario, tal como se indica en la Sección 2.1.

4.13. REQ013: Notificaciones Automáticas de Pago

Archivo: `tests/core/business/services/NotificationService.test.js` (3 casos), complementado por la verificación con *spy* dentro de `PagoService.test.js`.

Total de pruebas: 3 — **Estado:** 3/3 PASANDO (100 %).

Cómo se implementó

Listing 27: Cola de notificaciones con reintentos (`src/core/business/services/NotificationService.js`)

```
1 async procesarCola() {
2   if (this.procesando) return;
3   this.procesando = true;
4
5   while (this.cola.length > 0) {
6     const item = this.cola[0];
7     try {
8       item.intentos++;
9       await this.simularApiWhatsApp(item.telefono, item.mensaje);
10      this.cola.shift();
11    } catch (err) {
12      if (item.intentos >= 3) {
13        await pagoRepository.registrarAuditoria("
14          NOTIFICACION_WHATSAPP_FALLO_PERMANENTE", { telefono: item.
15            telefono, intentos: item.intentos });
16        this.cola.shift();
17      } else {
18        const esTest = process.env.NODE_ENV === 'test';
19        await new Promise(resolve => setTimeout(resolve, esTest ? 0 :
20          30000)); // RF-04: reintento cada 30s
21      }
22    }
23  }
24  this.procesando = false;
25 }
```

Listing 28: Verificación indirecta con `jest.spyOn` (`PagoService.test.js`)

```
1 const spyNotif = jest.spyOn(notificationService, 'enviarNotificacionPago');
2 const resultado = await pagoService.validarAprobarPago(pagoId, fechaReferencia);
3 // ...
4 expect(spyNotif).toHaveBeenCalled();
5 spyNotif.mockRestore();
```

Resultados Obtenidos

El archivo dedicado `NotificationService.test.js` ejercita tres escenarios de forma aislada: (i) registro de una notificación exitosa con tipo, destinatario, fecha e intentos; (ii) advertencia persistente cuando el copropietario no tiene teléfono registrado; y (iii) reintento de la cola hasta tres veces conservando el mensaje pendiente. Además, la prueba con `jest.spyOn` en `PagoService.test.js` confirma que `validarAprobarPago` invoca al servicio de notificaciones tras aprobar un pago. La entrega real por WhatsApp Business API queda fuera del alcance por requerir credenciales de despliegue.

4.14. Pruebas Transversales de Modelo de Dominio y Objetos de Valor

Archivos: `Usuario.test.js` (7), `Cedula.test.js` (7) y `Telefono.test.js` (7).

Total de pruebas: 21 — **Estado:** 21/21 PASANDO (100 %).

Estos tres componentes no pertenecen a un único requisito, sino que constituyen la infraestructura de validación reutilizada por REQ001 (formato de usuario y contraseña), REQ004 (validación de cédula/teléfono al importar) y REQ007 (validación al modificar copropietarios).

Cómo se implementó

Listing 29: Algoritmo Módulo 10 ecuatoriano (`src/core/domain/valueObjects/Cedula.js`)

```
1  validarCedulaEcuatoriana(cedula) {
2    if (cedula.length !== 10) return false;
3    const digitoRegion = parseInt(cedula.substring(0, 2), 10);
4    if ((digitoRegion < 1 || digitoRegion > 24) && digitoRegion !== 30) return
      false;
5
6    const ultimoDigito = parseInt(cedula.substring(9, 10), 10);
7    let suma = 0;
8    for (let i = 0; i < 9; i++) {
9      let multiplicador = (i % 2 === 0) ? 2 : 1;
10     let componente = parseInt(cedula.substring(i, i + 1), 10) * multiplicador;
11     if (componente > 9) componente -= 9;
12     suma += componente;
13   }
14   const digitoVerificador = (suma % 10 === 0) ? 0 : 10 - (suma % 10);
15   return digitoVerificador === ultimoDigito;
16 }
```

Para probar el caso de provincia “consular exterior” (30), fue necesario construir manualmente una cédula válida que superara el algoritmo, documentando el cálculo

directamente en el comentario de la prueba (ver también Sección 6.6):

Listing 30: Cédula exterior válida calculada manualmente (`Cedula.test.js`)

```
1 test('Debera permitir provincia exterior 30', () => {
2   // Dgitos: 3 0 0 0 0 0 0 1 2 -> Posiciones impares x2: 3*2=6, 0, 0, 0, 1*2=2
   // -> Suma = 8
3   // Posiciones pares x1: 0, 0, 0, 0 -> Suma = 0. Total = 8. Verificador: 10 - 8
   // = 2.
4   // 3000000012 es una cdula exterior vlida.
5   const ced = new Cedula('3000000012');
6   expect(ced.valor).toBe('3000000012');
7 });
```

Resultados Obtenidos

- **Cedula:** validación del algoritmo Módulo 10, limpieza de guiones/espacios, rechazo de provincia inválida y aceptación de la provincia exterior 30 — verificado.
- **Telefono:** normalización a formato internacional (+593...) desde los cuatro formatos de entrada aceptados (local con/sin cero, internacional con/sin signo +) y rechazo de números fijos — verificado.
- **Usuario:** longitud de usuario (3–20 caracteres), formato de correo, política de complejidad de contraseña (mayúscula, minúscula, número, carácter especial) y excepción para contraseñas ya cifradas con `bcrypt` (prefijo `$2b$`) — verificado.
- **Usuario.esBloqueado():** estado activo, bloqueo indefinido y bloqueo temporal vigente/expirado — verificado.

5. Resultados Obtenidos

5.1. Resumen de Ejecución

Fecha de ejecución: 23 de julio de 2026. **Ambiente:** Windows 11, Node.js, `jest` `-runInBand`. **Resultado:** TODOS LOS TESTS PASANDO.

Tabla 3: Resumen global de ejecución de la suite de pruebas

Métrica	Valor
Suites de prueba	17 / 17 (100 %)
Casos de prueba	86 / 86 (100 %)
Tiempo de ejecución	~5 s (posterior al primer arranque)
Requisitos con cobertura completa	12 / 13
Requisitos con cobertura parcial	1 / 13 (REQ006)
Snapshots	0

5.2. Desglose por Archivo

Tabla 4: Desglose de pruebas por archivo

Categoría	Archivo	Tests	Estado
Patrones de Diseño	<code>auditDecorator.test.js</code>	2	PASS
Patrones de Diseño	<code>excelAdapter.test.js</code>	4	PASS
Patrones de Diseño	<code>moraStrategy.test.js</code>	4	PASS
Servicios de Negocio	<code>AuthService.test.js</code>	9	PASS
Servicios de Negocio	<code>CopropietarioService.test.js</code>	7	PASS
Servicios de Negocio	<code>ExcelImportSample.test.js</code>	1	PASS
Servicios de Negocio	<code>NotificationService.test.js</code>	3	PASS
Servicios de Negocio	<code>PagoService.test.js</code>	10	PASS
Modelo de Dominio	<code>Usuario.test.js</code>	7	PASS
Objetos de Valor	<code>Cedula.test.js</code>	7	PASS
Objetos de Valor	<code>Telefono.test.js</code>	7	PASS
Datos (repositorio)	<code>sessionRepository.test.js</code>	3	PASS
Presentación (integración)	<code>authController.test.js</code>	4	PASS
Presentación (integración)	<code>copropietarioController.test.js</code>	5	PASS
Presentación (integración)	<code>pagoController.test.js</code>	4	PASS
Presentación (<i>middleware</i>)	<code>authMiddleware.test.js</code>	6	PASS
Presentación (vista)	<code>dashboard.test.js</code>	3	PASS
TOTAL		86	PASS

5.3. Métricas por Requisito

Tabla 5: Métricas de cobertura por requisito funcional

REQ	Nombre	Tests	Cobertura
REQ001	Gestión de Acceso y Registro de Usuarios	12	Completa
REQ002	Recuperación de Contraseña	4	Completa
REQ003	Configuración de Perfiles	6	Completa
REQ004	Importación de Copropietarios	8	Completa
REQ005	Consulta de Copropietarios	3	Completa
REQ006	Generación de Reportes	0*	Parcial
REQ007	Modificación de Copropietarios	3	Completa
REQ008	Eliminación de Copropietarios	3	Completa
REQ009	Registro de Pago de Expensa	4	Completa
REQ010	Validación de Pagos	5	Completa
REQ011	Cálculo de Sanción por Mora	10	Completa
REQ012	Generación de Comprobante de Pago	1	Completa
REQ013	Notificaciones Automáticas de Pago	3	Completa
Transversal	Modelo de Dominio / Objetos de Valor	21	Completa
Regresión	Vista <code>dashboard.html</code> (enlaces/ObjectId)	3	Completa
TOTAL		86	

* La restricción de reportes por rol se ejercita de forma indirecta a través de las pruebas de integración de controladores; `obtenerReportePagos` y `generarPdfReportePagos` no cuentan aún con un caso de prueba dedicado (ver Sección 7.4).

5.4. Distribución de Pruebas por Requisito

Tabla 6: Distribución porcentual de pruebas por requisito

REQ	Descripción breve	%	Proporción
REQ001	Acceso y Registro	14.0 %	██████████
REQ002	Recuperar Contraseña	4.7 %	███
REQ003	Perfiles	7.0 %	██████
REQ004	Importación	9.3 %	████████
REQ005	Consulta	3.5 %	██
REQ007	Modificación	3.5 %	██
REQ008	Eliminación	3.5 %	██
REQ009	Registro de Pago	4.7 %	████
REQ010	Validación de Pagos	5.8 %	██████
REQ011	Mora / Expensas	11.6 %	██████████
REQ012	Comprobante de Pago	1.2 %	█
REQ013	Notificaciones	3.5 %	██
Transversal	Dominio / Valor	24.4 %	████████████████
Regresión	Vista dashboard	3.5 %	██

6. Problemas y Soluciones

6.1. Recalculo del Recargo por Mora

Problema: la implementación original de `EcuandorianMora12Strategy` calculaba un interés *prorrateado diariamente* sobre una tasa anual del 12 % (`monto * (0.12/365) * diasRetraso`), lo cual no coincidía con la regla de negocio especificada: un recargo **fijo** del 12 % aplicado una sola vez por período vencido, independientemente de los días de atraso transcurridos.

Listing 31: Fórmula anterior (incorrecta respecto a la regla de negocio)

```
1 // Antes
2 calcularMora(monto, diasRetraso) {
3     if (diasRetraso <= 0) return 0;
4     const tasaDiaria = 0.12 / 365;
5     return parseFloat((monto * tasaDiaria * diasRetraso).toFixed(2));
6 }
```

Solución: se sustituyó la fórmula por un recargo fijo y se recalcularon a mano los valores esperados en `moraStrategy.test.js` y en la prueba de amortización FIFO de `PagoService.test.js` (de \$100.33 de deuda con mora a \$112.00, y del saldo pendiente resultante de \$50.33 a \$62.00), documentando la aritmética directamente en los comentarios de la prueba para que quede trazable ante futuras revisiones.

6.2. Ausencia de Generación de Deudas Mensuales

Problema: el repositorio `pagoRepository.createDeuda` existía, pero **ningún controlador ni servicio lo invocaba en producción**; solo se llamaba desde los propios archivos de prueba para fabricar datos de prueba manualmente. Esto significaba que, en un entorno real, jamás se generaba una deuda mensual real y la lógica de amortización FIFO nunca tenía un caso genuino que resolver.

Solución: se implementó `PagoService.generarExpensasMensuales`, expuesto mediante `POST /api/pagos/generar-expensas` (solo Administrador), y se añadieron 4 pruebas nuevas que verifican la generación, la no duplicación por período y el cálculo correcto del vencimiento (incluido el cambio de año en diciembre).

6.3. Redondeo de Punto Flotante en Cálculos Monetarios

Problema: las operaciones aritméticas con montos en JavaScript (`100 * 0.12`, `150.75 * 0.12`) pueden introducir errores de precisión de punto flotante (por ejemplo, `0.1 + 0.2 !== 0.3`), lo que provoca fallos de aserción intermitentes si no se redondea de forma explícita.

Solución: todos los cálculos monetarios se normalizan con `parseFloat(valor.toFixed(2))` antes de compararse, tanto en el código de producción (`moraStrategy.js`, `PagoService.js`) como en las aserciones de prueba, garantizando que las comparaciones sean deterministas a dos decimales.

6.4. Determinismo Temporal en Pruebas de Mora

Problema: `validarAprobarPago` calcula los días de atraso comparando la fecha de vencimiento contra la fecha actual (`new Date()`); si la prueba dependiera de la fecha real del sistema, el resultado (días de atraso, y por lo tanto la lista de deudas vencidas) cambiaría cada vez que se ejecutara la suite en un día distinto, produciendo pruebas *no deterministas*.

Solución: el método acepta un parámetro opcional `fechaReferencia`, que las pruebas fijan explícitamente (`new Date('2026-07-20')`) para congelar el cálculo de días de atraso y obtener siempre el mismo resultado, independientemente del día real en que se ejecute la suite.

6.5. Verificación de un Efecto Secundario Asíncrono en Segundo Plano

Problema: `NotificationService.enviarNotificacionPago` encola el mensaje y dispara `procesarCola()` **sin esperar su resolución** (`this.procesarCola();` sin `await`), de modo que el envío real ocurre en segundo plano después de que la prueba ya continuó su ejecución. Un `await` directo sobre el resultado del envío no reflejaría el comportamiento real del sistema.

Solución: en lugar de esperar el efecto secundario completo, se utilizó `jest.spyOn(notificationService, 'enviarNotificacionPago')` para verificar únicamente que el método de encolado fue invocado por `validarAprobarPago`, que es la responsabilidad que le corresponde al servicio de pagos; la entrega real por WhatsApp queda fuera del alcance de esa prueba (ver también la brecha de cobertura de REQ013 en la Sección 4.13).

6.6. Construcción de Datos de Prueba Válidos para el Algoritmo Módulo 10

Problema: para probar el caso de la provincia “consular exterior” (código 30) del algoritmo de validación de cédula ecuatoriana, se requería un número de 10 dígitos que superara el dígito verificador Módulo 10; generar uno al azar y ajustarlo por prueba y error hubiera sido poco confiable y difícil de mantener.

Solución: se calculó manualmente un valor válido (3000000012) aplicando el propio algoritmo paso a paso, y ese cálculo se documentó como comentario dentro de la prueba

(ver Sección 4.14), de modo que cualquier persona que revise el test pueda verificar la aritmética sin tener que ejecutar el código.

6.7. Tiempo de Arranque de `mongodb-memory-server`

Problema: la primera ejecución de la suite tras una instalación limpia del proyecto tarda considerablemente más (~18s) que las ejecuciones posteriores, porque `mongodb-memory-server` debe descargar y almacenar en caché el binario de MongoDB correspondiente al sistema operativo.

Solución: no se requirió ningún cambio de código; se documenta el comportamiento esperado para que no se interprete como una regresión de rendimiento. En ejecuciones subsecuentes, con el binario ya cacheado localmente, el tiempo total baja a ~5 segundos.

7. Conclusiones

7.1. Logros Alcanzados

- 100 % de los casos de prueba existentes pasando (86/86) en 17 suites.
- 12 de 13 requisitos funcionales del backlog con cobertura automatizada completa; el restante (REQ006, generación de reportes) cuenta con cobertura parcial documentada y trazable.
- Corrección verificada de una regla de negocio crítica (recargo de mora fijo del 12 %) mediante pruebas de regresión.
- Incorporación de una funcionalidad ausente (generación de expensas mensuales) junto con su suite de pruebas correspondiente.
- Suite de ejecución rápida y reproducible (~5 s en caliente).

7.2. Calidad del Software

Confiabilidad (5/5): 0 pruebas intermitentes observadas en múltiples corridas; resultados reproducibles gracias al aislamiento por `beforeEach` y a fechas de referencia fijas en los cálculos de mora.

Cobertura de Requisitos (4/5): 12 de 13 requisitos con cobertura completa; se documenta explícitamente la única brecha restante (REQ006) en lugar de omitirla.

Mantenibilidad (5/5): pruebas organizadas por capa arquitectónica (dominio/objetos de valor, servicios, patrones, presentación), con comentarios que documentan la aritmética esperada en los casos financieros.

Rendimiento (5/5): suite completa en segundos, apta para ejecutarse en cada cambio antes de integrar código nuevo.

7.3. Lecciones Aprendidas

1. **Las reglas de negocio financieras deben verificarse contra la especificación, no solo contra el código existente:** el error de la fórmula de mora (interés prorratedo vs. recargo fijo) habría pasado inadvertido si las pruebas solo hubiesen validado que el código hiciera lo que ya hacía.
2. **Una funcionalidad puede estar “implementada” en el repositorio de datos sin estar realmente disponible en producción** si ningún controlador o servicio la invoca; las pruebas de integración end-to-end (aunque sea a nivel de servicio) ayudan a detectar estas rutas muertas.
3. **El determinismo temporal es esencial en dominios financieros:** cualquier cálculo que dependa de `new Date()` debe aceptar una fecha de referencia inyectable para que las pruebas sean reproducibles indefinidamente.

4. **No toda dependencia externa debe mockearse igual:** aislar por completo con `jest.mock()` es adecuado para middlewares de control de flujo, pero para lógica que depende de consultas relacionales reales, una base de datos en memoria produce pruebas más representativas del comportamiento en producción.
5. **Documentar la aritmética esperada dentro del propio test** (como se hizo en los casos de mora y del algoritmo Módulo 10) reduce el costo de mantenimiento futuro de las pruebas.

7.4. Recomendaciones Futuras

1. **Cerrar la brecha de cobertura restante:** añadir pruebas dedicadas para `obtenerReportePagos/generarPdfReportePagos` (REQ006), única línea del backlog con cobertura sólo indirecta. Como refuerzo, ampliar `NotificationService.test.js` para ejercitar el registro de auditoría por fallo permanente de la cola de WhatsApp.
2. **Pruebas de integración HTTP adicionales:** extender `pagoController` con `supertest` para verificar la carga real de comprobantes vía `multer` (tipo de archivo, límite de 5 MB) y el flujo completo de aprobación/rechazo por HTTP.
3. **Cobertura de código automatizada:** incorporar `-coverage` en el script de `npm test` y fijar un umbral mínimo por carpeta (dominio, servicios, presentación) para detectar regresiones de cobertura en el tiempo.
4. **Pruebas end-to-end de la interfaz visual:** usar Playwright o Cypress para validar el flujo completo desde `login.html` y `dashboard.html` hasta la API, incluyendo la expiración de sesión por inactividad.
5. **Integración continua (CI/CD):** automatizar la ejecución de `npm test` en cada *pull request* mediante GitHub Actions.
6. **Pruebas de mutación:** evaluar herramientas como Stryker para medir la calidad real de las aserciones existentes, más allá del simple conteo de casos que pasan.

7.5. Resumen Final

El plan de pruebas unitarias de AliGest cubre, con 86 casos automatizados distribuidos en 17 suites, el núcleo de la lógica de negocio del sistema: autenticación y control de acceso, administración de copropietarios, y el ciclo completo de pagos, mora, auditoría y notificaciones. Durante la elaboración de este informe se detectó y corrigió una discrepancia real entre la fórmula de mora implementada y la regla de negocio especificada, y se incorporó la funcionalidad de generación de expensas mensuales que faltaba en el flujo de producción, ambas respaldadas por pruebas de regresión. La única brecha de cobertura restante (la invocación directa de los reportes financieros, REQ006) se documenta de forma explícita como trabajo pendiente, en lugar de presentarse como resuelta.

Estado final: APROBADO CON OBSERVACIONES — el núcleo funcional crítico (autenticación, copropietarios, pagos, mora, comprobantes y notificaciones) está completamente cubierto y verificado; sólo la invocación directa de los reportes financieros (REQ006) requiere una prueba adicional antes de considerarse con cobertura total.

Anexos

Anexo A: Comandos de Ejecución

Listing 32: Comandos útiles para ejecutar la suite

```
1 # Ejecutar todos los tests (segn package.json: jest --runInBand)
2 npm test
3
4 # Ejecutar un archivo de prueba especifico
5 npx jest tests/core/business/services/PagoService.test.js
6
7 # Ejecutar con salida detallada por caso
8 npx jest --verbose
9
10 # Ejecutar un caso puntual por nombre (coincidencia de texto)
11 npx jest -t "amortizacin FIFO"
```

Anexo B: Estructura de Prueba Típica

Listing 33: Patrón Arrange-Act-Assert utilizado en todo el proyecto

```
1 describe('Componente - Funcionalidad', () => {
2   beforeEach(async () => {
3     // Arrange: preparar datos (fixtures) y limpiar la base de datos en
      memoria
4   });
5
6   test('Debe hacer algo especifico', async () => {
7     // Arrange: datos particulares del caso
8     const datos = { /* ... */ };
9
10    // Act: ejecutar la funcionalidad bajo prueba
11    const resultado = await servicio.metodo(datos);
12
13    // Assert: verificar el resultado esperado
14    expect(resultado).toBe(/* valor esperado */);
15  });
16 });
```

Anexo C: Configuración Completa

Listing 34: jest.config.js

```
1 module.exports = {
2   testEnvironment: 'node',
```

```
3   verbose: true,  
4   setupFilesAfterEnv: ['./tests/setup.js'],  
5   testTimeout: 30000  
6 };
```

Listing 35: Fragmento relevante de `package.json`

```
1 {  
2   "name": "aligest-prototype",  
3   "version": "1.0.0",  
4   "scripts": {  
5     "start": "node app.js",  
6     "test": "jest --runInBand"  
7   },  
8   "dependencies": {  
9     "bcrypt": "^5.1.1",  
10    "express": "^4.19.2",  
11    "mongoose": "^9.7.4",  
12    "multer": "^2.2.0",  
13    "pdfkit": "^0.19.1",  
14    "xlsx": "^0.18.5"  
15  },  
16  "devDependencies": {  
17    "jest": "^30.4.2",  
18    "mongodb-memory-server": "^11.2.0",  
19    "supertest": "^7.2.2"  
20  }  
21 }
```

Listing 36: Mecanismo de expiración de sesión por inactividad (RNF-01), `authMiddleware.js`

```
1 const SESION_MAX_INACTIVIDAD_MS = 2 * 60 * 60 * 1000; // 2 horas  
2  
3 function verificarSesion(req, res, next) {  
4   const role = req.headers['x-user-role'];  
5   const userId = req.headers['x-user-id'];  
6   if (!role) {  
7     return res.status(401).json({ error: "No autorizado. Inicie sesin para  
8       acceder a AliGest." });  
9   }  
10  if (role !== 'ADMINISTRADOR' && role !== 'COPROPIETARIO') {  
11    return res.status(403).json({ error: "Acceso denegado. Rol invlido o  
12      desconocido." });  
13  }  
14  
15  const lastActivity = parseInt(req.headers['x-last-activity'], 10);
```

```

14     if (!isNaN(lastActivity) && Date.now() - lastActivity >
        SESION_MAX_INACTIVIDAD_MS) {
15         return res.status(401).json({
16             error: "Sesin expirada por inactividad. Por favor, inicie sesin
                nuevamente.",
17             sesionExpirada: true
18         });
19     }
20
21     req.user = { role, id: userId || null };
22     next();
23 }

```

Anexo D: Métricas Detalladas

Tabla 7: Distribución de pruebas por categoría arquitectónica

Categoría	Tests	%
Modelo de dominio y objetos de valor	21	24.4 %
Servicios de negocio	30	34.9 %
Patrones de diseño (Strategy/Decorator/Adapter)	10	11.6 %
Datos (repositorio de sesiones)	3	3.5 %
Presentación (<i>middlewares</i> , controladores y vistas)	22	25.6 %
TOTAL	86	100 %

Tabla 8: Distribución de pruebas por tipo de validación

Tipo de validación	Tests aproximados
Creación y validación de objetos de dominio	21
Reglas de negocio y validación cruzada contra BD	30
Control de acceso y expiración de sesión	13
Patrón Decorator/Strategy/Adapter	10
Integración HTTP (<i>supertest</i>)	9
Regresión de vista y serialización de enlaces (<i>dashboard.html</i>)	3

FIN DEL INFORME

Documento generado el 23 de julio de 2026

Sistema AliGest — Plan de Pruebas Unitarias

Requisitos REQ001–REQ005, REQ007–REQ013 y los bloques transversal (dominio/objetos de valor) y de regresión de vista: cobertura completa (86 pruebas en 17 suites). Sólo REQ006 mantiene cobertura parcial documentada.